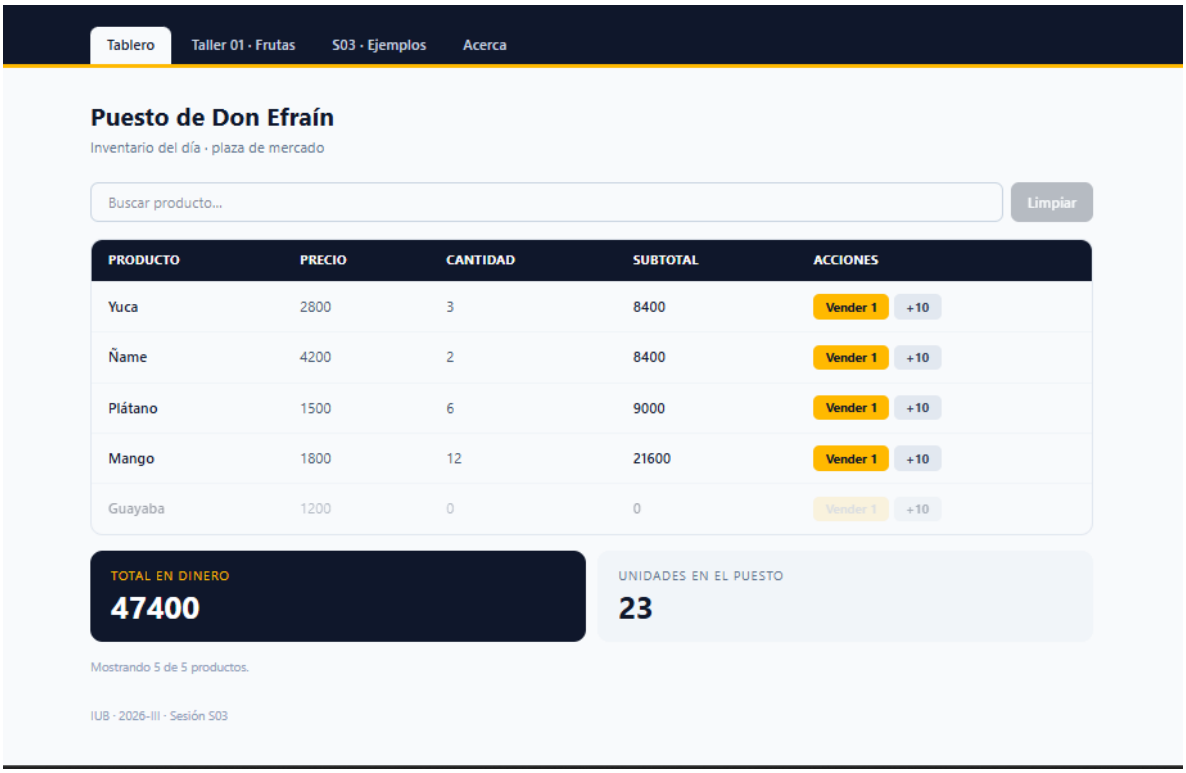
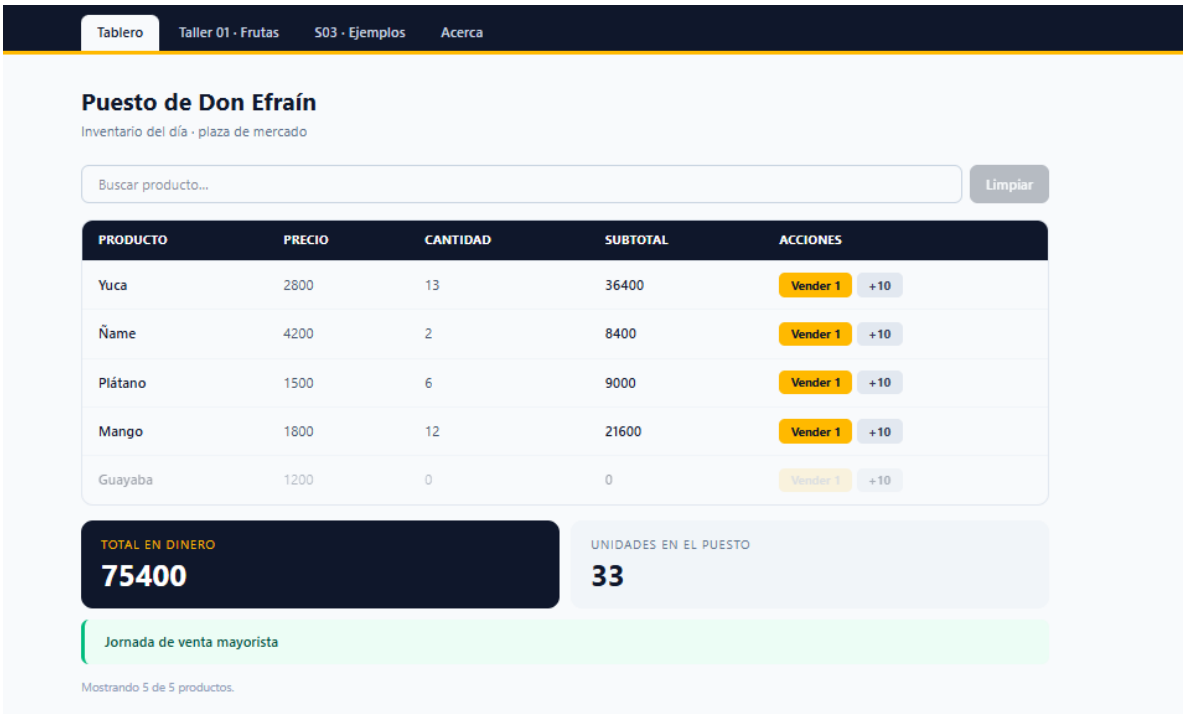
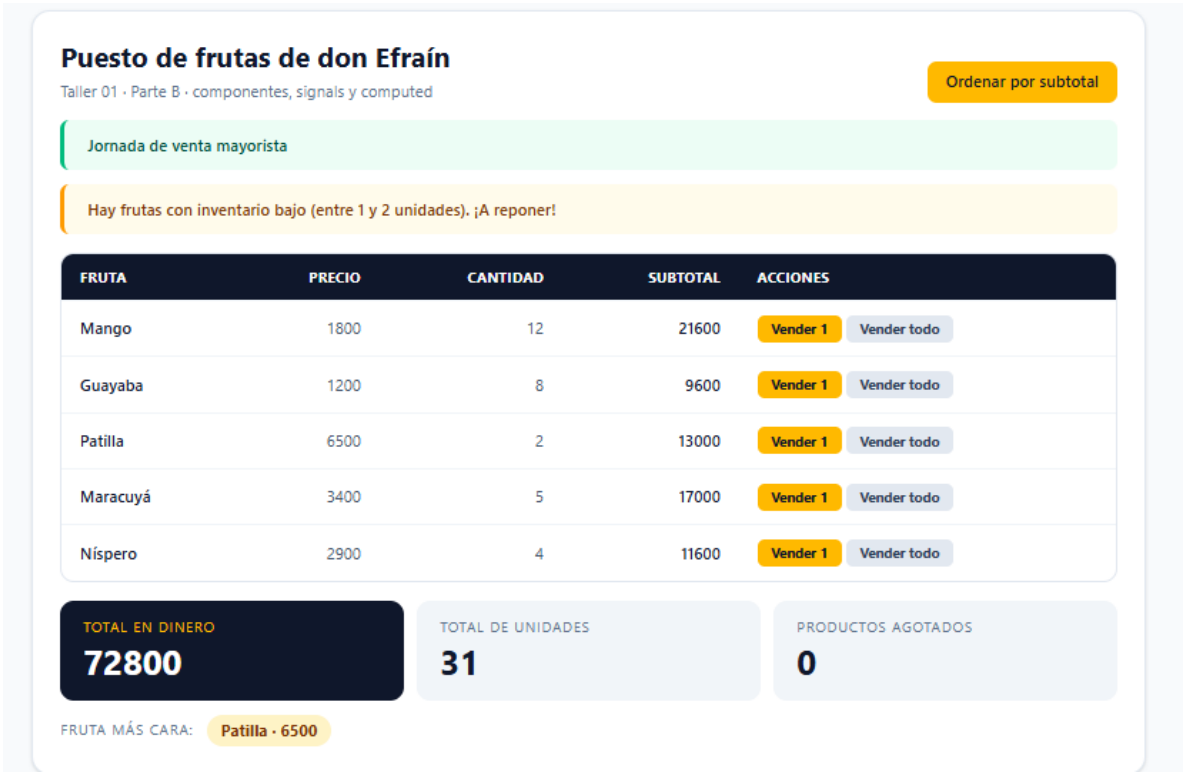
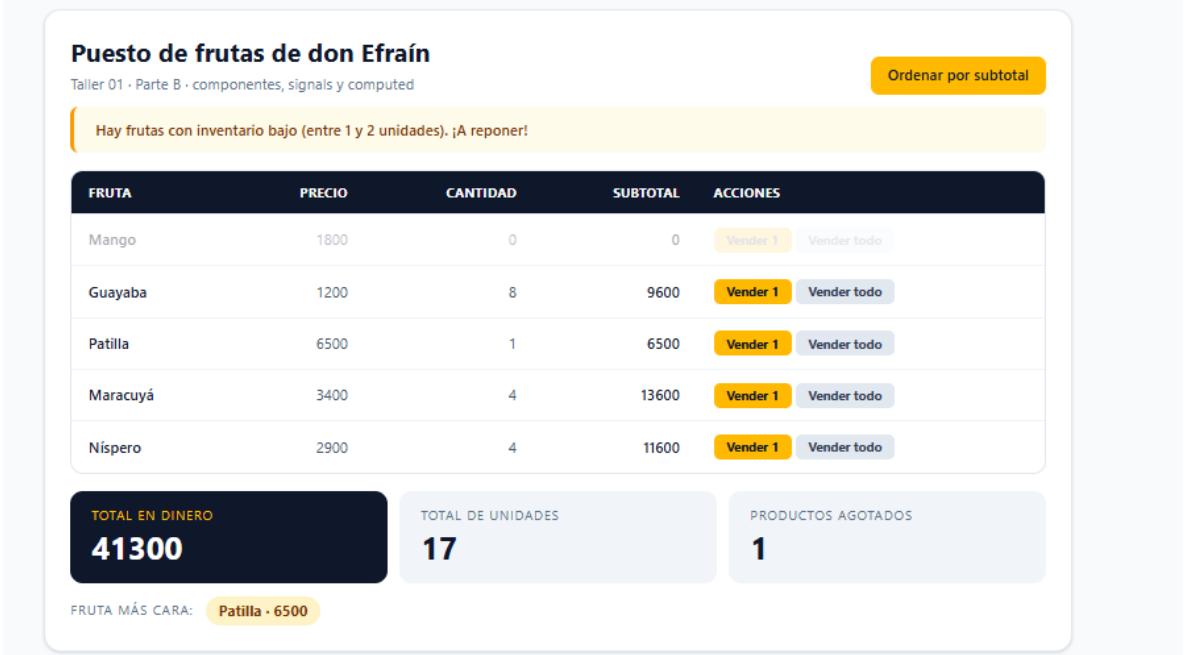

Tablero
Taller 01 · Frutas
S03 · Ejemplos
Acerca
Puesto de Don Efraín
Inventario del día · plaza de mercado
Buscar producto...
Limpiar
PRODUCTO
PRECIO
CANTIDAD
SUBTOTAL
ACCIONES
Yuca
2800
3
8400
Vender 1
+10
Ñame
4200
2
8400
Vender 1
+10
Plátano
1500
6
9000
Vender 1
+10
Mango
1800
12
21600
Vender 1
+10
Guayaba
1200
0
0
Vender 1
+10
TOTAL EN DINERO
47400
UNIDADES EN EL PUESTO
23
Mostrando 5 de 5 productos.
IUB · 2026-III · Sesión S03


Tablero
Taller 01 · Frutas
S03 · Ejemplos
Acerca
Puesto de Don Efraín
Inventario del día · plaza de mercado
Buscar producto...
Limpiar
PRODUCTO
PRECIO
CANTIDAD
SUBTOTAL
ACCIONES
Yuca
2800
13
36400
Vender 1
+10
Ñame
4200
2
8400
Vender 1
+10
Plátano
1500
6
9000
Vender 1
+10
Mango
1800
12
21600
Vender 1
+10
Guayaba
1200
0
0
Vender 1
+10
TOTAL EN DINERO
75400
UNIDADES EN EL PUESTO
33
Jornada de venta mayorista
Mostrando 5 de 5 productos.


Puesto de frutas de don Efraín
Taller 01 · Parte B · componentes, signals y computed
Ordenar por subtotal
Jornada de venta mayorista
Hay frutas con inventario bajo (entre 1 y 2 unidades). ¡A reponer!
FRUTA PRECIO CANTIDAD SUBTOTAL ACCIONES
Mango 1800 12 21600 Vender 1 Vender todo
Guayaba 1200 8 9600 Vender 1 Vender todo
Patilla 6500 2 13000 Vender 1 Vender todo
Maracuyá 3400 5 17000 Vender 1 Vender todo
Níspero 2900 4 11600 Vender 1 Vender todo
TOTAL EN DINERO
72800
TOTAL DE UNIDADES
31
PRODUCTOS AGOTADOS
0
FRUTA MÁS CARA: Patilla · 6500


Puesto de frutas de don Efraín
Taller 01 · Parte B · componentes, signals y computed
Ordenar por subtotal
Hay frutas con inventario bajo (entre 1 y 2 unidades). ¡A reponer!
FRUTA PRECIO CANTIDAD SUBTOTAL ACCIONES
Mango 1800 0 0 Vender 1 Vender todo
Guayaba 1200 8 9600 Vender 1 Vender todo
Patilla 6500 1 6500 Vender 1 Vender todo
Maracuyá 3400 4 13600 Vender 1 Vender todo
Níspero 2900 4 11600 Vender 1 Vender todo
TOTAL EN DINERO
41300
TOTAL DE UNIDADES
17
PRODUCTOS AGOTADOS
1
FRUTA MÁS CARA: Patilla · 6500



1 · Qué desapareció

En la Parte A escribí a mano cuatro tipos de cosas que en la Parte B ya no existen:

- La manipulación del DOM a mano. En frutas.html tuve que crear cada celda con document.createElement('td'), asignarle .textContent, y reconstruir tbody con innerHTML = '' antes de cada repintado. Eso lo reemplazan los bindings de la plantilla: la fila se declara una vez con {{ f.nombre }}, {{ subtotal(f) }}, y Angular se encarga de sincronizar el DOM.
- Los listeners manuales. El tbody.addEventListener('click', ...) con delegación, y la responsabilidad de que ese listener exista siempre, lo reemplaza el evento de plantilla (click)="vender(f.nombre)". Angular se suscribe por mí y limpia el listener cuando el componente se destruye; no me toca acordarme.
- El repintado manual. La función pintar() que tenía que volver a llamar después de cada cambio (con su nota mental: "acuérdate de repintar") lo reemplaza computed. El total en dinero no se "recalcula llamando a una función": se deriva de la signal frutas() y se actualiza solo cuando esa signal cambia de verdad.

- Las clases CSS a mano. En la Parte A marcaba la fila agotada con tr.classList.add('agotada') y, si luego había venta, tenía que recordar quitarla. Eso lo reemplaza un binding condicional: [class.opacity-40]="f.cantidad === 0". La condición se declara una vez y Angular pone o quita la clase cuando la expresión cambia.

2 · El requisito 5, lado a lado

En la Parte A el mensaje me costó seis líneas repartidas en tres lugares: el con su regla display:none para que arrancara oculto (dos líneas), y dentro de pintar() un if (dinero > 50000) { … display='block' } else { display='none' } (tres líneas) — y encima tenía que estar seguro de que pintar() se ejecutara después de cada venta, porque si la olvidaba, el mensaje se quedaba con el valor viejo.

En la Parte B el mismo comportamiento es una sola línea: @if (esMayorista()) { … mensaje … }. La diferencia se debe a que en la Parte A mostrar u ocultar es una acción que yo debía ejecutar cada vez; una responsabilidad imperativa y dispersa. En la Parte B es una condición que Angular evalúa contra el estado: la condición esMayorista() es un computed que depende de totalDinero(), que a su vez depende de frutas(); si el total cruza los 50 000, Angular crea el nodo; si baja, lo destruye. No hay lugar donde "olvidarse", porque no hay un paso manual que recordar.

3 · El bug que ya no se puede escribir

En la Parte A el error clásico era: cambio el dato (f.cantidad--), me olvido de llamar a pintar(), y el dato quedó modificado por dentro mientras la pantalla muestra el valor viejo. No hay ningún error; los dos mundos (dato y pantalla) se mantienen unidos solo por mi disciplina.

En la Parte B eso es estructuralmente imposible por dos razones. Primera: la plantilla no guarda una copia aparte del dato; se dibuja directamente leyendo la signal. Segunda: si intento el atajo de la Parte A, this.frutas()[0].cantidad--, la signal compara el valor nuevo por identidad (la referencia en memoria del arreglo), no por contenido: como el arreglo es el mismo, la signal cree que nada cambió y no notifica — o sea, ese código *hace invisiblemente nada*, no deja un dato "modificado a medias". El único camino para que la pantalla cambie es entregar un valor nuevo con .update() y un arreglo construido con map/…, y ese camino notifica automáticamente a quien lee la signal. De modo que el "cambié el dato pero la

pantalla no se actualizó" ya no puede existir: o el cambio es real y se propaga solo, o no es un cambio válido y no sucede nada.

4 · Lo que costó más

Honestamente, lo que más tiempo me tomó no fue la sintaxis — los paréntesis de las signals, el @for con track, los corchetes de los bindings se aprenden rápido porque son mecánicos y el compilador avisa cuándo faltan. Lo que de verdad costó fue la lógica de la inmutabilidad. Traía el hábito de "modificar lo que ya existe" (push, cantidad--, sort directo, cambiarle una propiedad a un objeto), y aquí esa costumbre es justo lo que rompe la aplicación. El salto mental fue dejar de pensar "muevo la cantidad del mango en el arreglo" y pasar a pensar "entrego un arreglo nuevo igual, pero con el mango teniendo una unidad menos". Me costó también recordar leer cada signal con sus paréntesis dentro de las propias funciones, cosa que en la Parte A no existía. La lección que me llevo es que la sintaxis se resuelve con práctica y con el error del compilador a la vista, pero la lógica se resuelve entendiendo primero *cuál es el valor nuevo que debe reemplazar al viejo*; una vez que uno razona eso, traducirlo a map y ... es casi automático.